User Manual

COMP3211 Group 149 PIM | Handoff edition | 20 October 2026

This application manages plain-text notes, tasks, events and contacts in one offline terminal session. It supports creating, editing, searching, displaying, deleting, saving and explicitly loading records. The local verified environment is Python 3.12.6 on macOS 15.7.3. Windows instructions are supplied but a real Windows verification run is outstanding.

Start and finish

Open a terminal at the source root:

```
python3.12 -X utf8 -B main.py
```

Windows target command, not yet verified on Windows:

```
py -3.12 -X utf8 -B main.py
```

The prompt is pim>. Enter application commands there, one per line, without copying the prompt itself. The collection starts empty even when sample or previous .pim files exist. There is no numbered menu and no automatic loading or saving. Use exit to finish; unsaved changes trigger a confirmation workflow.

Command conventions

Command words and field/type spellings are case-sensitive. Use update as the editing command; edit, help, quit and textual search aliases are not supported. Quoted values use double quotes. Write an embedded quote as \" and a literal backslash as \\. Other escapes such as \n or \t are not accepted as escapes. Values already containing actual permitted internal whitespace retain it.

```
create PLAIN_TEXT text="A \"quoted\" note in C:\\notes"
save "C:/Work/PIM data/notes.pim"
```

Paths with spaces must also be quoted. They are relative to the source-root working directory unless absolute. Windows backslashes must be doubled inside quotes; forward slashes may be used. No shell-variable or home-directory expansion is performed inside the application.

Record IDs are positive ASCII decimal integers. Leading zeroes refer to the same number. New IDs exceed all IDs successfully allocated or loaded in the current session; deleted high IDs are not reused. IDs and types cannot be edited.

Create records

Supply each required business field exactly once; assignment order is flexible. Missing, repeated or extra fields reject the command. These examples assume a fresh empty session and create IDs 1 through 4:

```
create PLAIN_TEXT text="Quick note"
create TASK description="Report" deadline="2026-11-20 20:00"
create EVENT description="Review" start="2026-11-22 18:00" alarm="2026-11-22 17:00"
create CONTACT name="Alex" address="Sample Street" mobile="+0012345"
```

Type	Required business fields
PLAIN_TEXT	text
TASK	description, deadline
EVENT	description, start, alarm
CONTACT	name, address, mobile

Text must be single-line Unicode. Outer whitespace is removed; internal spaces, tabs and underscores are preserved. Empty or whitespace/underscore-only text is invalid. Line breaks are rejected, including at the ends.

Mobile is text: an optional single leading + followed by 1-15 ASCII digits. Leading zeroes are preserved. Internal spaces and non-ASCII digits are invalid. The program does not check whether a number or address exists.

Times use exact YYYY-MM-DD HH:MM, local 24-hour time, years 0001-9999, valid calendar dates, hours 00-23 and minutes 00-59. No seconds/timezone suffix or automatic timezone conversion is accepted. Past times are allowed.

An Event's alarm must be earlier than or equal to start, during both creation and update. Alarms are stored, displayed, searchable and saved fields. They do not trigger notifications. This is the approved provisional scope, still subject to instructor confirmation; there is no scheduler.

Duplicate creation

Records are duplicates when type and every normalized business field are identical, comparing text case-sensitively and excluding ID. The program warns with matching IDs, then asks y/n. y creates a separate new ID; n adds nothing and consumes no ID. Invalid responses produce ERROR and repeat the confirmation. Updating or loading equal business contents does not prompt for duplicates.

Update, show and delete

```
update 2 deadline="2026-11-25 23:59"
update 4 mobile="000123"
show 2
show all
delete 4
```

update changes exactly one business field of the selected record. The complete candidate must remain valid. For example, moving an Event start earlier than its alarm is rejected without changing either field. ID/type edits and fields belonging to another record type fail. A normalized no-op update returns the same value without introducing unsaved changes. Invalid IDs/values preserve the original collection and its existing dirty state.

show ID displays one complete record; show all displays every record in ascending numerical ID order, so 2 precedes 10. Every record includes ID, type and all business fields. Dates always use four-digit years and minute precision. Example output:

```
ID: 2
Type: TASK
description: Report
deadline: 2026-11-25 23:59
```

An empty show all reports Collection is empty. A missing ID reports ERROR, and show all can be used to discover valid IDs.

delete first displays the selected record, then asks y/n. Only y removes it; n leaves it intact. Invalid confirmation responses repeat without mutation. Only the selected record is deleted. Deletion retains session ID history.

Expected errors and recovery

Errors begin with ERROR and explain the affected operation/input and relevant expected format or lookup guidance. Examples include invalid deadline format, noneditable fields, unknown criteria and unreadable files. Ordinary errors appear on standard output and return to a usable prompt. Submit a corrected command; partially valid creation or update requests do not commit.

```
create TASK description="Bad date" deadline="2026-02-29 12:00"
search unknown ~ "x"
show 999
```

The first date is invalid because 2026 is not a leap year. The second field is unknown; it is not treated as an absent field. There is no reserved :cancel field-entry convention. Cancellation uses the documented confirmation prompts.

Search expressions

search returns complete current records in ascending numerical ID order. Zero matches is explicit; no count is required for nonempty results.

Operation	Syntax and meaning
Type	type = TASK; exact PLAIN_TEXT, TASK, EVENT or CONTACT
Text	text ~ "word"; consecutive, case-sensitive literal substring
Time	deadline < "2026-11-22 00:00"; also > or =
Negation	!condition
And / or	condition && condition; condition || condition
Grouping	(condition)

Text fields are text, description, name, address and mobile. description applies to Task and Event. Time fields are deadline, start and alarm. Comparisons bind first, then !, then &&, then disjunction. Parentheses override precedence. Only symbols are operators; CONTAINS, AND, OR and NOT are not alternatives. ~ does not interpret patterns, regular expressions or wildcards.

Search substrings must be nonempty and are never trimmed. Spaces and underscores are valid substrings, unlike underscore-only stored text. Operator characters inside quotes remain literal.

```
search text ~ " _ "
search type = TASK && description ~ "COMP3211"
search type = TASK && (description ~ "report" || description ~ "tests")
search type = TASK && !(deadline < "2026-11-20 20:00")
search alarm = "2026-11-22 17:00"
```

A condition on a field absent from a record is false. Negation applies after that result: a Contact can match !(deadline < "2026-11-20 20:00"). Add type = TASK when contacts and other types should be excluded. An unknown field is an error for the entire expression.

The whole expression is validated before any records are evaluated, including branches that could be skipped. For example, search type = TASK || unknown ~ "x" is rejected even if Tasks exist. Unsupported operators, incompatible fields, empty literals, invalid dates, incomplete quotes/escapes and trailing tokens also fail without returning a partial result set.

Define and reuse criteria

```
define type = TASK && description ~ "COMP3211"
criteria
search @C1
```

Successful definitions receive C1, C2, etc. The identifier and original expression are displayed; criteria lists them in numerical C-ID order. No criteria defined is explicit when empty. Definitions are immutable; define a changed expression as a new criterion. Invalid definitions consume no ID. Saved criteria evaluate current records after create/update/delete/load. They survive successful loads but are not written to .pim files and end with the session. A new run starts with no criteria.

Save and load

```
save "my_records.pim"
load "my_records.pim"
load "samples/empty.pim"
```

Use a lowercase .pim suffix and an existing, accessible parent directory. The format is UTF-8 JSON v1 without BOM. Save writes the entire collection, including empty collections, in numerical ID order. It preserves IDs and every business field; criteria, dirty state and the old session counter are excluded. Do not hand-edit files unless you understand the strict schema.

An existing destination triggers overwrite y/n. Refusing leaves the file, records and unsaved-change state unchanged; it is not a successful save. Save success is reported only after staged writing and replacement complete. On failure the live collection is retained and temporary-file cleanup is attempted. There is no autosave or backup/recovery subsystem.

Load validates the entire file before replacing the current collection. It does not merge records or renumber IDs. A valid empty collection replaces all records with none. Distinct saved IDs with equal business contents are kept. Duplicate keys/IDs, unsupported versions, missing/extra fields, invalid values, encoding errors and non-canonical saved values reject the entire file.

Successful loading retains session criteria and may raise, but never lowers, the current session's ID high-water. A failed load preserves the old collection, criteria, counter and dirty state, except that an already completed preliminary save remains completed and leaves the old collection clean.

Unsaved changes and confirmation choices

Create, delete and content-changing updates mark the collection dirty. Reads, searches, criterion definitions, no-op edits, errors and refusals do not change dirty. Startup and successful save/load establish a clean state. Before dirty load or exit:

Response	Meaning
s	Request a quoted save path; continue only after saving succeeds
d	Discard and continue; during load, retain old data until the new file validates
c	Cancel this load/exit and remain in the session

Invalid responses repeat the same prompt. Failed saving or declined overwrite stops the pending action and returns to main input. Retry save explicitly, or issue load/exit again and separately choose d if appropriate.

Discard + invalid load leaves old records dirty. Successful preliminary save + invalid load leaves old records present and clean. A cancelled exit leaves the session active. Normal exit is distinct from exceptional EOF termination.

Interruption and permanently closed input

Ctrl-C at an input/confirmation/path boundary cancels pending uncommitted input and returns to the prompt. It does not silently discard records, grant consent or undo an independently completed save. Forced process termination cannot preserve unsaved memory.

If stdin permanently closes, the application does not keep prompting. When dirty it writes this diagnostic to standard error and exits with status 1:

```
ERROR Standard input is closed; terminating the session with unsaved changes.
```

It does not auto-save, overwrite, report success or pretend the user chose discard. Clean EOF may exit with status 0 without claiming EOF saved anything. This exceptional rule is the explicitly approved clarification now included in the handoff specification copies; ordinary errors still use standard output.

Worked example using the supplied samples

In a fresh session:

```
load "samples/design_scenario.pim"
define type = TASK && description ~ "COMP3211" && deadline < "2026-11-22 00:00"
search @C1
update 2 deadline="2026-11-25 23:59"
search @C1
show 7
save "updated_tasks.pim"
exit
```

Before the update, IDs 2 and 7 match. Afterwards only 7 matches; record 7 is unchanged. The save is real; a preexisting updated_tasks.pim requires overwrite confirmation. Choose n to protect it or y to replace it intentionally.

samples/mixed.pim contains all four types with IDs 1,2,7,10,11 and illustrative data, including Unicode and a mobile with +/leading zeroes. samples/empty.pim is a valid empty collection, not an empty byte file. Samples never load automatically. After loading mixed.pim in a fresh session, the next successful creation receives ID 12.

Keep important data in an explicitly saved file before ending the process. Windows remains a release verification task; consult the Developer Manual and Windows checklist. Concurrent external file changes and power-loss recovery are outside the current guarantees.